Authentication Flow

Overview

Authentication is JWT-based. The system supports email or phone registration, is
sues long-lived access and refresh tokens, then enriches authenticated requests
with role and permission context from MongoDB.

Current implementation

Registration

- route: `POST /api/auth/register`
- accepts email or phone, username, password, location, community selection, and
 country
- validates country and selected communities
- hashes password with bcrypt
- creates the user and joins them to one or more communities

Login

- route: `POST /api/auth/login`
- resolves a user by identifier
- verifies password
- returns access and refresh tokens

Request authentication

- middleware: `middleware/auth.js`
- requires `Authorization: Bearer <token>`
- verifies token with `ACCESS_TOKEN_SECRET`
- loads the user record from MongoDB
- checks revocation timestamps such as `sessionRevokedAt` and `accessTokenRevoke
dAt`
- computes rank and permissions via `getPermissions(...)`

Important modules

- `routes/auth.js`
- `middleware/auth.js`
- `middleware/permissions.js`
- `models/user.model.js`

Known issues

- access and refresh token expiry are both currently long-lived
- auth middleware logs verbose token diagnostics, which is useful operationally
but noisy
- role evaluation depends on multiple legacy and current fields (`role`, `roleNa
me`, `accessRole`, `staffRole`, `publicRoles`)

Scaling concerns

- every authenticated request performs a MongoDB user lookup
- permission enrichment is request-time rather than token-time

Recommended improvements

1. reduce token lifetimes and tighten refresh rotation
2. centralize role derivation into a single canonical resolver
3. separate debug logging from normal authentication flow